

PRODUCT REQUIREMENTS DOCUMENT

SplitSmart

Itemized, receipt-level bill splitting for a P2P payments app

Product area	Consumer Payments — Peer-to-Peer / Bill Splitting
Document type	Feature PRD (v1 — MVP scope)
Author	Reza Rahid Sami, Product Manager (candidate portfolio)
Status	Draft for review
Framing	A new feature within a Venmo-style payments app

This PRD accompanies a working prototype (a four-step scan → review → assign → split flow with AI-powered receipt parsing). It reframes that build as the feature a payments team would ship, and is written to sit alongside the prototype in a portfolio.

Contents

1. Summary

Splitting a bill among friends is one of the most common reasons people open a P2P payments app. But when a group orders different things at a restaurant, today's tools force a choice between two bad options: split the total *evenly* (so the person who had a salad subsidizes the person who had steak and three cocktails), or make one person do the mental math line by line and then type each amount in manually.

SplitSmart adds **itemized, receipt-level splitting** to the app. A user photographs the receipt; an AI vision layer parses it into line items, tax, and tip; the user confirms and corrects the parse, taps who shared each item, and the app distributes tax and tip **proportionally** to what each person ordered — then generates a payment request for each person automatically.

WHY IT MATTERS

Even splitting is already solved. Fair, by-item splitting is not — and it's the case that causes the most friction and the most awkward “you actually owe a bit more” conversations. SplitSmart targets exactly that gap, using receipt parsing to remove the manual data-entry that has made itemized splitting too tedious to do inside a payments app.

2. Problem & context

2.1 What the app already does today

This is a feature inside an existing payments app, so the PRD starts from what already ships. Two capabilities are relevant:

- **Split by amount / evenly.** Users can split a charge evenly across a group or enter custom dollar amounts per person.
- **Shared-expense tracking.** Groups can track ongoing shared costs (trips, roommates) with running tallies and a “settle up” that nets balances down to the fewest transactions.

Both split by *amount*. Neither splits an actual *receipt by item*. That is the gap.

2.2 The gap: nobody splits by item

To split a restaurant bill fairly today, a user has to read the receipt, mentally total each person's items, work out each person's share of tax and tip, and then type those amounts into the app one at a time — or leave the app entirely for a separate bill-splitting tool and copy the results back. That friction is why most groups just split evenly and quietly accept that it's unfair.

2.3 Who feels the pain

- **The person who paid (primary):** fronted the whole bill and now has to figure out and collect what everyone owes, fairly, without nickel-and-diming friends.
- **The lighter spender (secondary):** ordered a salad and a water, and doesn't want to pay an equal quarter of a table that included premium entrées and a bar tab.
- **The group (relationship stakes):** bill-splitting is socially awkward; a tool that makes fairness effortless removes friction from the friendship, which is the emotional core of why P2P apps exist.

3. Goals & non-goals

3.1 Goals

- **Make fair, by-item splitting faster than splitting evenly by hand** — the feature has to be less work than the unfair shortcut it replaces.
- **Remove manual entry** by parsing the receipt automatically.
- **Distribute tax and tip fairly** in proportion to each person's items, with no manual math.
- **Generate the payment requests** so collecting is one tap, not a series of manual charges.
- **Keep the user in control of correctness** with an explicit review step before any money is involved.

3.2 Non-goals (out of scope for v1)

- **Auto-charging without confirmation.** The feature drafts requests; the user reviews and sends. Nothing moves money automatically.
- **Replacing even-split or group tracking.** Itemized splitting is an added path, not a replacement for existing flows.
- **Full receipt-history storage / expense analytics.** v1 splits a bill in the moment; it doesn't build a spending ledger.
- **Non-restaurant use cases (groceries, bills).** The MVP is tuned for the restaurant receipt, the highest-frequency case.

4. User & proposed workflow

4.1 Primary persona

PERSONA — JORDAN, THE ONE WHO PAID

Just covered a \$180 dinner for four on their card. Three friends had wildly different orders. Jordan wants everyone to pay their fair share without being the person squinting at a receipt doing arithmetic at the table, and without feeling cheap. Today Jordan either eats the difference or spends five minutes on mental math and manual requests. Wants it done in under a minute, and fairly.

4.2 Target workflow

1. **Scan** — Jordan photographs the receipt. The AI vision layer extracts line items, prices, tax, and tip into a structured list.
2. **Review** — the parsed items appear in an editable list. Jordan fixes any misread item or price before continuing. Nothing depends on the parse being perfect.
3. **Assign** — Jordan taps who shared each item. Shared items (a bottle of wine, an appetizer) split evenly among whoever is tapped.
4. **Split** — the app computes each person's item subtotal, distributes tax and tip proportionally, and shows a per-person total with a clear breakdown.
5. **Request** — Jordan sends the generated payment requests in one tap. Each friend gets a request for their exact fair share.

5. Solution & requirements

5.1 Receipt parsing (the enabling capability)

AI vision turns a receipt photo into structured data: an array of line items with prices, plus tax and tip. This is the capability that makes itemized splitting fast enough to be worth doing — it removes the manual entry that has historically made by-item splitting too tedious for a payments app. Quantities are folded into item names; subtotal and total lines are ignored in favor of the individual items.

5.2 The review step (a requirement, not a nicety)

Parsed items are shown in a fully editable list before assignment. The user can correct any item name, fix a price, add a missed item, or delete a spurious one. This step is mandatory in the flow and exists for a specific reason: AI receipt parsing is imperfect on faded, crumpled, or oddly formatted receipts, and the product must never silently split money based on a misread. Putting a human checkpoint between the parse and any money movement is the core trust guarantee of the feature.

5.3 Assignment & proportional distribution

- **Per-item assignment.** Each item is assigned to one or more people; a shared item splits evenly among those tapped.
- **Proportional tax & tip.** Rather than splitting tax and tip evenly, each person's share is scaled to their portion of the item subtotal — the fair treatment, computed automatically.
- **Live, transparent totals.** Each person's total shows the item / tax / tip breakdown so the split is explainable, not a black box.

5.4 Request generation

Once the split is set, the app generates a payment request per person for their exact share, addressed to the person who paid. Sending is one tap and always user-initiated.

5.5 Scope: MVP vs. later

Capability	MVP (v1)	Later
Scan receipt → AI-parsed items	Yes	—
Editable review / correction step	Yes	—
Per-item assignment + shared items	Yes	—
Proportional tax & tip	Yes	—
Generate per-person requests	Yes	—
Manual entry fallback (no photo)	Yes	—
Natural-language assignment ("I had the salad")	—	v2
Save/split history & receipts	—	v2
Non-restaurant receipts (groceries)	—	Future
Real-time collaborative assigning	—	Future

6. Success metrics

6.1 Product metrics (the goal)

Metric	Definition	Target direction
Itemized-split adoption	Share of splits done by item vs. evenly	Up
Time to complete a split	Scan → requests sent	Down
Split completion rate	Started splits that end in sent requests	Up
Request fulfillment rate	Generated requests that get paid	Up / hold
Repeat usage	Users who split by item more than once	Up

6.2 Quality / trust metrics (leading indicators)

- **Parse-correction rate** — how often users edit the parsed items (a proxy for parse accuracy and where to improve it).
- **Review-step drop-off** — abandonment at review signals parsing is too unreliable or the step too heavy.
- **Post-send edits / disputes** — requests corrected after sending; a signal that splits are landing wrong.

7. Key tradeoffs & decisions

7.1 AI parsing vs. reliability — solved with a human checkpoint

Receipt parsing is the feature's enabler and its biggest risk: open-ended photos of real receipts parse imperfectly. Rather than chase perfect accuracy or avoid AI, the design accepts imperfect parsing and inserts a mandatory review step where the user corrects mistakes before any money is involved. This converts a reliability problem into a small, bounded UX step, and it's the right call because the cost of a silent error (charging a friend the wrong amount) is far higher than the cost of a quick confirmation.

7.2 Proportional vs. even tax and tip

Tax and tip could be split evenly (simpler) or proportionally to each person's items (fairer). The feature chooses proportional, because fairness is the entire reason a user reaches for itemized splitting in the first place — splitting the items fairly but the tip evenly would reintroduce the unfairness the feature exists to remove. The math is invisible to the user, so the fairness costs them nothing.

7.3 Scan-first, with a manual fallback

The flow leads with scanning because that's the magic, but always allows manual entry. This protects the experience when a receipt won't parse (a handwritten total, a photo that fails) and keeps the feature usable in the case where there's no paper receipt at all.

7.4 An added path, not a replacement

Itemized splitting is offered alongside even-split, not instead of it. Even-split is genuinely the right tool sometimes (four people, four identical coffees). Forcing everyone through item assignment would add friction where none is needed. The product should route the user to the lightest tool for their case.

8. Prototype & honest limitations

A working prototype implements the full flow: scan, AI parse, editable review, per-item assignment, proportional tax/tip, and per-person request generation. Receipt parsing runs on an AI vision model called through a small server-side proxy (so the API key is never exposed to the browser). The following limitations are stated deliberately — they are scoped choices, not oversights:

- **Parsing is imperfect on messy receipts.** Faded, angled, or unusually formatted receipts parse unevenly. The review step is the mitigation; improving parse accuracy is a fast-follow, measurable via the parse-correction rate.
- **No real payments.** The prototype generates and displays requests but does not move money; it demonstrates the product flow, not a production payments integration.
- **No persistence or accounts.** Splits aren't saved between sessions — appropriate for a prototype, out of scope for v1's core loop.
- **Restaurant-tuned.** Parsing and the flow assume a restaurant receipt; other receipt types are future scope.

PRODUCTION CONSIDERATION SURFACED BY THE BUILD

The prototype's receipt scanning calls the AI model through a server-side proxy rather than from the browser, because an API key in client code would be exposed to anyone who views the page. This is the correct production pattern too: any real deployment routes the vision call through a backend that holds the credential. Flagging this is part of the feature's real engineering shape, not an afterthought.

9. Rollout plan

6. **Internal dogfood.** Ship behind a flag to internal users; gather real receipts to measure parse-correction rate and find failure modes before customers see them.
7. **Limited beta.** Release to a small opt-in cohort. Measure time-to-split and completion vs. the existing even-split flow, and watch review-step drop-off.
8. **Tune parsing on real data.** Use correction data to improve the parse prompt/model on the receipts users actually photograph.
9. **Phased GA.** Roll out by cohort, watching the product metrics in Section 6, with even-split and manual entry always available as fallbacks.
10. **Fast-follow (v2).** Natural-language assignment and split history, prioritized by observed demand.

10. Risks & mitigations

Risk	Mitigation
AI misreads the receipt and someone is charged wrongly	Mandatory editable review step before any request is generated; nothing is sent without user confirmation
Parsing too unreliable to feel magical	Manual-entry fallback always available; parse accuracy improved on real correction data before GA
Feature adds friction where even-split was fine	Itemized splitting offered as an added path; user routed to the lightest tool for their case

Risk	Mitigation
API key exposure via client code	Vision call routed through a server-side proxy; key never ships to the browser
Social discomfort (feels like nickel-and-diming)	Framing and UX emphasize fairness and speed; the person who paid stays in control of what's sent
Privacy of receipt images	Images used only to parse the split in-session; no receipt storage in v1

11. Open questions

- What parse-accuracy bar is “good enough” to lead with scanning vs. nudging users toward manual entry?
- Should tax/tip proportionality be user-overridable for groups that prefer to split those evenly?
- Does receipt image handling raise data-retention or privacy requirements that shape the backend design?
- Is there demand to extend beyond restaurants (groceries, shared household bills) that would change the parsing target?

Prepared as a portfolio PRD accompanying a working prototype. “Venmo” is referenced only to frame the product context; this is an independent concept, not affiliated with or endorsed by any company.